Write steps for Install and configure Antivirus software on system.

install and Configure Antivirus Software in Your System

We will install **Quick Heal Antivirus** using the Quick Heal CD/DVD or by downloading the installer from the official website.

Part A: Installation

1. Insert the **Quick Heal Antivirus CD/DVD** into the DVD drive or download the installer from the official website.
2. Open the setup file by double-clicking it.
3. Click on the **Install** button.
4. Accept the license agreement and follow the on-screen instructions.
5. Wait for the installation process to complete successfully.

Part B: Registration

1. Open **Quick Heal Antivirus** after installation.
2. On the dashboard, click **Register Now**.
3. Enter the **20-digit Product Key** and click **Next**.
4. Fill in the required details such as:
    - Name
    - Email Address
    - Contact Number
    - Country, State, and City
5. Verify the entered information and click **Next** to complete the registration.

Part C: Configuration

1. Activate the antivirus software successfully.
2. Perform an **initial full system scan**.
3. Select the **scan frequency** such as daily or weekly.
4. Choose the **type of scan** such as Quick Scan or Full Scan.
5. Click **Finish** to save the settings and apply protection.

Result

Quick Heal Antivirus is successfully **installed, registered, activated, and configured** on the system.

Write steps for Perform Backup and Restore of the System

Write Steps for Perform Backup and Restore of the System

Part A: Backup of the System

1. Open **Control Panel** and go to **Backup and Restore**.
2. Click on **Set up backup**.
3. Select the drive or external storage device where the backup will be saved.
4. Choose whether to let Windows select the files automatically or select them manually.
5. Click **Next**.
6. Set the **backup schedule** (daily, weekly, or monthly).
7. Click **Save settings and run backup**.
8. Wait until the backup process is completed successfully.

Part B: Restore of the System

1. Open **Control Panel** and select **Backup and Restore**.
2. Click on **Restore my files** or **Restore all users' files**.
3. Browse and select the backup file or folder.
4. Choose the location where the files need to be restored.
5. Click **Restore**.
6. Wait for the restore process to complete.
7. Verify that the files or system data have been restored properly.

Result

The system backup and restore process is performed successfully.

Write steps for Setup passwords to operating systems and applications

Write Steps for Setup Passwords to Operating Systems and Applications

Part A: Set Password for Operating System

1. Open **Control Panel** or **Settings**.
2. Go to **Accounts / User Accounts**.
3. Select **Sign-in options** or **Create password**.
4. Enter the **new password**.
5. Re-enter the password to confirm it.
6. Add a **password hint** if required.
7. Click **Save / Apply**.
8. Restart the system and verify the password at login.

Part B: Set Password for Application

1. Open the required application.
2. Go to **Settings / Security / Preferences**.
3. Select the **Password or Lock option**.
4. Enter the new password.
5. Confirm the password by re-entering it.
6. Save the settings.
7. Close and reopen the application to verify the password protection.

Result

Passwords are successfully set for the operating system and applications.

Write steps for Apply security to file folder or application using access permissions and verify

Write Steps for Apply Security to File, Folder or Application Using Access Permissions and Verify

Part A: Apply Security / Access Permissions

1. Select the **file, folder, or application** on which security is to be applied.
2. Right-click on it and choose **Properties**.
3. Open the **Security** tab.
4. Click on **Edit** to change permissions.
5. Select the **user or group name**.
6. Set the required permissions such as:
   - **Read**
   - **Write**
   - **Modify**
   - **Full Control**
7. Click **Apply** and then **OK** to save the changes.

Part B: Verify Permissions

1. Log in with the selected user account.
2. Try to **open, edit, delete, or modify** the file/folder/application.
3. Check whether the access is allowed or denied according to the permissions set.
4. Confirm that the security settings are working correctly.

Result

Security is successfully applied to the file, folder, or application using access permissions and verified.

Write a program to implement Caesar Cipher with example c program

```
#include <stdio.h>
```

```
#include <string.h>

int main() {
    char text[100];
    int key, i;

    printf("Enter the plaintext: ");
    scanf("%s", text);

    printf("Enter the key value: ");
    scanf("%d", &key);

    for(i = 0; text[i] != '\0'; i++) {
        if(text[i] >= 'A' && text[i] <= 'Z') {
            text[i] = ((text[i] - 'A' + key) % 26) + 'A';
        }
        else if(text[i] >= 'a' && text[i] <= 'z') {
            text[i] = ((text[i] - 'a' + key) % 26) + 'a';
        }
    }

    printf("Cipher Text: %s", text);

    return 0;
}
```

Write a Program to implement Vernam Cipher with example

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char plaintext[100], key[100], cipher[100];
    int i, len;

    printf("Enter plaintext (uppercase only): ");
    scanf("%s", plaintext);

    printf("Enter key (same length as plaintext): ");
    scanf("%s", key);

    len = strlen(plaintext);

    for(i = 0; i < len; i++) {
        cipher[i] = ((plaintext[i] - 'A') + (key[i] - 'A')) % 26 + 'A';
    }

    cipher[len] = '\0';

    printf("Cipher Text: %s", cipher);

    return 0;
}
```

Write an algorithm to implement Caesar Cipher with example

Algorithm

1. Start
2. Input the **plaintext**

3. Input the **key value (shift)**
4. Read each character of the plaintext one by one
5. Shift each character by the key value using the formula:

Cipher = (Plaintext + Key) mod 26

6. Replace the original character with the shifted character
7. Repeat until all characters are encrypted
8. Display the **cipher text**
9. Stop

Example

Plaintext: HELLO
Key: 3

Encryption

- H → K
- E → H
- L → O
- L → O
- O → R

Cipher Text

KHOOR

Write a program to implement Rail fence technique with example

```
#include <stdio.h>
#include <string.h>

int main() {
```

```
    char text[100];
    int i;

    printf("Enter the plaintext: ");
    scanf("%s", text);

    printf("Cipher Text: ");

    /* First rail - even index characters */
    for(i = 0; text[i] != '\0'; i += 2) {
        printf("%c", text[i]);
    }

    /* Second rail - odd index characters */
    for(i = 1; text[i] != '\0'; i += 2) {
        printf("%c", text[i]);
    }

    return 0;
}
```

Write a program to implement simple columnar transposition technique with example

```
#include <stdio.h>
#include <string.h>

int main() {
    char text[100];
```

```
    int key, i, j, len, rows;

    printf("Enter plaintext: ");
    scanf("%s", text);

    printf("Enter key (number of columns): ");
    scanf("%d", &key);

    len = strlen(text);
    rows = (len + key - 1) / key;

    printf("Cipher Text: ");

    for(i = 0; i < key; i++) {
        for(j = i; j < len; j += key) {
            printf("%c", text[j]);
        }
    }

    return 0;
}
```

Write a Algorithm to implement simple columnar transposition technique with example

Algorithm

1. **Start**
2. Input the **plaintext message**
3. Input the **key value** (number of columns)
4. Find the **length of the plaintext**

5. Calculate the **number of rows** required using:
   Rows = Length / Key
6. If the plaintext does not fit exactly, add **blank spaces or dummy characters**
7. Arrange the plaintext **row-wise in a matrix**
8. Write the characters from **left to right** in each row
9. After filling the matrix, read the characters **column-wise from top to bottom**
10. Store the read characters as **cipher text**
11. Display the **cipher text**
12. **Stop**

Example

Plaintext: HELLOWORLD
Key: 4

Step 1: Arrange row-wise

H E L L
O W O R
L D

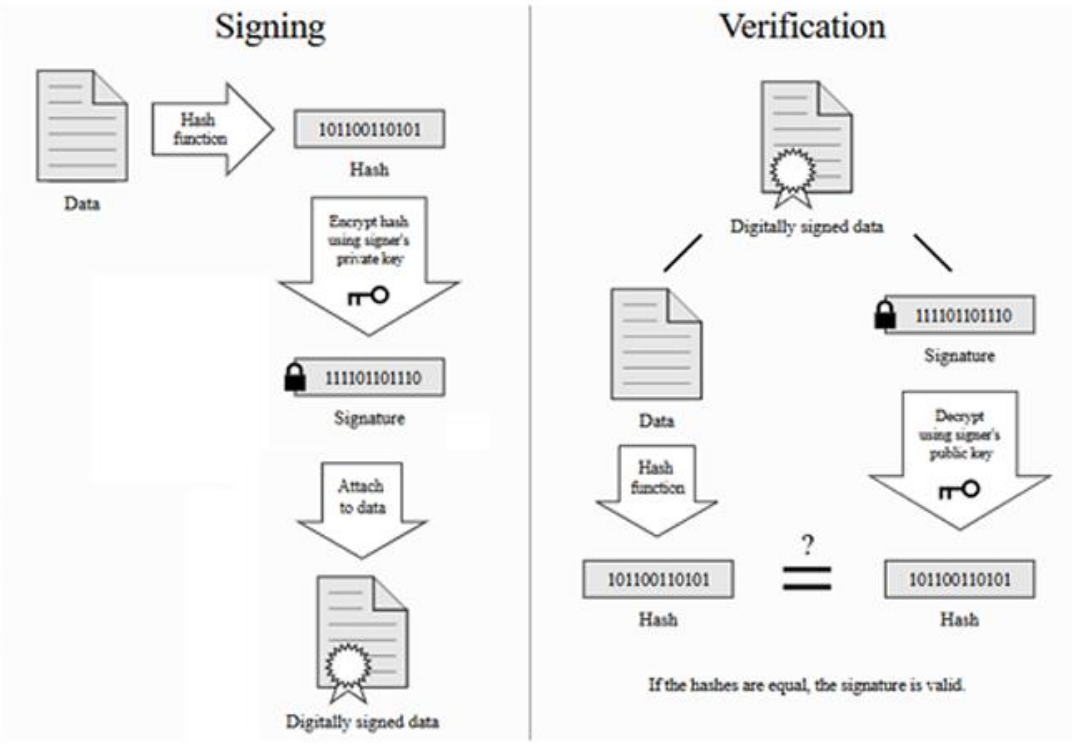
Step 2: Read column-wise

- Column 1 → H O L
- Column 2 → E W D
- Column 3 → L O
- Column 4 → L R

Cipher Text

HOLEWDLOLR

Write details about digital signature



→ A digital signature is a cryptographic mechanism that ensures authentication, integrity, and non-repudiation for electronic messages or documents.

The working of a digital signature involves:

- Hash Functions
- Message Digest
- Asymmetric Key Cryptography (Public & Private key)

---

1. key generation

Before creating a digital signature, the sender generates a key pair:

- **Private key** → used for creating the DS
- **Public key** → shared with others for verifying the signature

This step establishes authenticity and trust.

---

2. Digital Signature Creation – Signing Process

Step 1: Generation of Message Digest

The sender applies a hash function to the original message (eg: SHA-256)

Produces a fixed-length output called MD

Represents the unique identity of the message

Step 2: Encryption using Sender's Private key

The sender encrypts the message digest using his private key

The encrypted digest is the Digital Signature

Step 3: Sending Data

The sender sends both:

- Plaintext Message
- Encrypted Message Digest (DS)

Signing phase is now complete.

---

3. Digital Signature Verification – Receiver Side

Step 1: Receiver Generates Message Digest

The receiver applies the same hash function to the received message

Generates a new message digest.

Step 2: Decrypting the Signature

The receiver decrypts the digital signature using the sender's public key

This produces the original message digest created by the sender

---

Step 3: Comparison for Verification

The receiver compares:

- Digest generated from the received message
- Digest obtained after decrypting the signature

Decision:

If Message digest match → Signature Valid

If Message digest doesn't match → Invalid

Write details about steganography with diagram,encode and decode the message .

Write Details About Steganography with Diagram, Encode and Decode the Message

Definition

Steganography is the technique of **hiding secret information inside another file or medium** such as an image, audio, video, or text so that the existence of the message is not visible to others.
Unlike cryptography, which changes the message into unreadable form, steganography hides the **presence of the message itself**. (Kaspersky India)

Diagram

Simple Block Diagram

Secret Message + Cover Image

↓

Embedding Process

↓

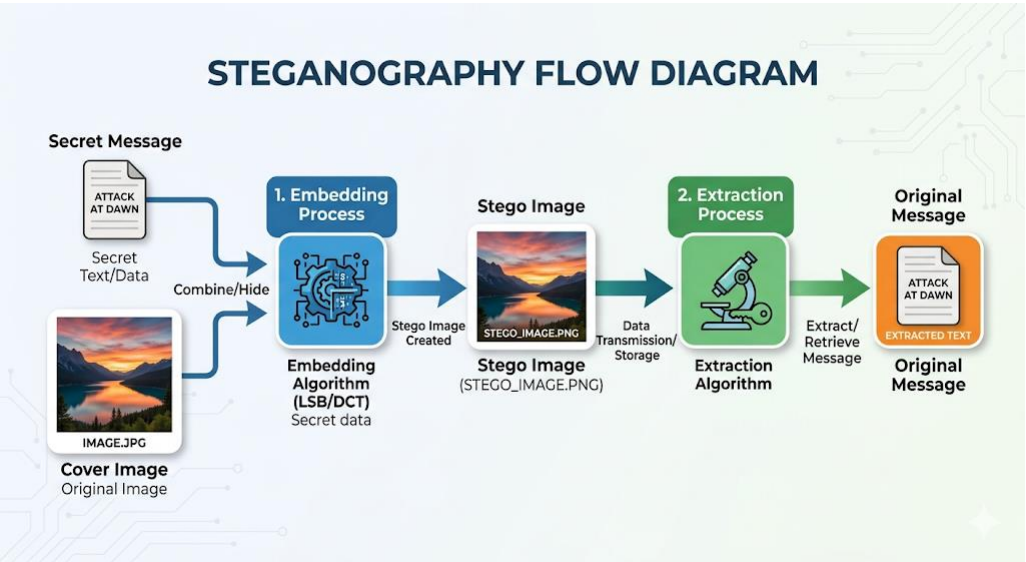
Stego Image

↓

Extraction Process

↓

Original Message



Encoding Process

1. Select a **cover file** (usually an image).
2. Take the **secret message** to be hidden.
3. Use a steganography tool such as **OpenStego**.

4. Apply the **embedding algorithm**.
5. The tool hides the message inside the cover image.
6. The output file is called the **Stego Image/File**. (EC-Council)

Decoding Process

1. Open the **Stego Image/File**.
2. Use the **extract/decode option** in the tool.
3. Provide the correct **password/key** if used.
4. The tool reads the hidden bits and recovers the message.
5. The **original secret message** is displayed. (PMC)

Example

Secret Message: HELLO

- During **encoding**, the word HELLO is hidden inside an image.
- During **decoding**, the hidden word is extracted back as HELLO.

Applications

- Secure communication
- Military data transfer
- Confidential document sharing
- Digital watermarking (Kaspersky India)

Result

The secret message is successfully **encoded and decoded using steganography**.

i. Grant security to file, folder or application using access permissions and verify it
ii. Grant access permission while sharing file and folder

i. Grant security to file, folder or application using access permissions and verify it

Steps

1. Select the **file, folder, or application**.
2. Right-click and choose **Properties**.
3. Open the **Security** tab.
4. Click **Edit** to change permissions.
5. Select the required **user or group**.
6. Set the permissions such as:
    - **Read**
    - **Write**
    - **Modify**
    - **Full Control**
7. Click **Apply** and then **OK**.
8. Log in with that user account and verify whether access is **allowed or denied** as per the permissions.

Result

Security is successfully granted and verified.

---

ii. Grant access permission while sharing file and folder

Steps

1. Select the **file or folder** to be shared.
2. Right-click and select **Properties**.
3. Open the **Sharing** tab.
4. Click on **Share / Advanced Sharing**.
5. Select the users or group with whom the file/folder is to be shared.
6. Set the permission type:
    - **Read**
    - **Read/Write**
7. Click **Share** and then **Done**.
8. Verify the shared access from another system or user account.

Result

Access permission is successfully granted while sharing the file and folder.

Create and verify Hash Code for given message using any Open-source tool (Example-Cryptool)

Steps to Create Hash Code

1. Open **CrypTool** software.
2. Click on **File** → **New**.
3. Enter the given message in the text area.
   Example: **HELLO WORLD**
4. Go to **Encrypt/Decrypt** → **Hash**.
5. Select the hashing algorithm such as **MD5 / SHA-1 / SHA-256**.
6. Click **Generate / OK**.
7. The software will display the **hash code (digest)** for the message.

Steps to Verify Hash Code

1. Copy the generated hash value.
2. Enter the same message again in CrypTool.
3. Generate the hash using the **same algorithm**.
4. Compare both hash values.
5. If both values are the same, the message is **verified successfully**.
6. If the message changes even by one character, the hash value changes completely.

Example

Message: HELLO

SHA-256 Hash:
3733cd977ff8eb18b987357e22ced99f46097f31ecb239e878ae63760e83e4d5

Result

Hash code is successfully created and verified using CrypTool.

Use Steganography to encode and decode the message using any open-source tool (Example-OpenStego)

Step-by-Step Procedure

Step 1: Download and Install OpenStego

- Visit the official website: https://www.openstego.com
- Download the version compatible with your operating system.
- Install the software by following the on-screen instructions.

Step 2: Launch OpenStego

- Open the **OpenStego application** after installation.
- The main interface window will appear.

Step 3: Select the File to Embed Data

- Click on the **Embed** option.
- Under **Cover File**, select the image or file in which the data is to be hidden.
- This file is called the **cover file**.

Step 4: Select the Data to Embed

- Under **Message File**, select the file containing the secret message or data to be hidden.
- It may be a text file, image, or any other file.

Step 5: Configure Security Settings

- Enter a **password/key** for encryption if required.
- Choose the **output file name and location**.

Step 6: Embed the Data

- Click on the **Embed** button.
- OpenStego creates a new **stego file** containing the hidden data.
- A confirmation message will appear.

Step 7: Extract the Hidden Data

- Click on the **Extract** option.
- Select the **stego file**.
- Choose the output location.
- Click **Extract**.

Step 8: Verify Extracted Data

- Open the extracted file from the selected folder.
- Verify that the hidden message is recovered successfully.

Result

The secret message is successfully **encoded and decoded using OpenStego**

Create and verify digital signature using any Open- source tool (Example-Cryptool)

Aim

To create and verify a **digital signature** using **CrypTool**.

Steps to Create Digital Signature

1. Open **CrypTool** software.
2. Click on **File** → **New** and type the message.
   Example: HELLO WORLD
3. Go to **Digital Signatures / Sign** option.
4. Select the **signature algorithm** such as **RSA**.

5. Generate or select the **private key**.
6. Click on **Sign**.
7. The software will create the **digital signature** for the message.

Steps to Verify Digital Signature

1. Open the signed message in **CrypTool**.
2. Select **Digital Signatures / Verify**.
3. Choose the sender's **public key**.
4. Click on **Verify**.
5. CrypTool compares the message hash with the decrypted signature hash.
6. If both values match, the signature is **valid**.
7. A confirmation message such as **Signature Verified Successfully** will be displayed.

Example

Message: HELLO

After signing, a digital signature is generated.
On verification, the result shows: **Valid Signature**

Result

The digital signature is successfully **created and verified using CrypTool**.